CELEBAL SUMMER INTERNSHIP (CSI)

DATA ENGINEERING TRACK — WEEK 8 ASSIGNMENT REPORT

E-Commerce Order Analytics System

An End-to-End Data Analytics Pipeline using Python, Pandas, SQL and Streamlit

---

Prepared By	Raj Kumar
Institution	Poornima University, Jaipur
Program	B.Tech CSE (AI & Data Science), Final Year
Internship	Celebal Technologies — Data Engineering Intern
Assignment	Week 8 — E-Commerce Order Analytics System
GitHub Repository	https://github.com/Raj-Kumar-Sharma/CEI-Assignment-8

1. Project Objective

This project delivers an end-to-end data analytics system that processes and analyzes e-commerce order data using Python and SQL. It covers the complete data lifecycle: generating realistic transactional datasets with intentional real-world inconsistencies, cleaning and validating that data using Pandas, loading it into a relational SQL warehouse while preserving referential integrity, and deriving business insights through complex SQL — joins, aggregations, window functions, common table expressions (CTEs), and cohort retention analysis. The system exposes these insights through both a command-line reporting tool and an interactive Streamlit dashboard.

2. Scope of Work

- Generate synthetic e-commerce datasets (customers, products, orders, order items) with deliberately injected data-quality issues.
- Clean and validate the raw data using Pandas — handling invalid emails, malformed dates, orphaned foreign keys, and negative/zero quantities.
- Design a normalized relational schema and load cleaned data into a SQL warehouse (SQLite, with a pluggable Snowflake backend).
- Write complex analytical SQL: multi-table joins, aggregations, window functions (NTILE, ROW_NUMBER), CTEs, and cohort retention logic.
- Build a CLI reporting tool that generates dynamic executive summaries with data-integrity audits.
- Build an interactive multi-page Streamlit dashboard for visual exploration of the same metrics.
- Handle critical edge cases — referential integrity cascades, discount/quantity anomalies, and date-parsing failures — to ensure robustness.

3. Technology Stack

Language	Python 3.12
Data Processing	Pandas
Synthetic Data	Faker
Database	SQLite (default) / Snowflake (pluggable)
SQL Techniques	Joins, Aggregations, Window Functions, CTEs, Cohort Analysis
Dashboard	Streamlit + Plotly
Testing	Pytest
Version Control	Git & GitHub

4. System Architecture & Workflow

The system follows a four-stage pipeline architecture, where each stage feeds the next and both the CLI reporting tool and the Streamlit dashboard draw from a single shared analytics core, ensuring that business logic (such as the revenue formula and which order statuses are excluded from calculations) is defined exactly once.

Data Generation → ETL Cleaning & Validation → SQL Warehouse → CLI Reports / Streamlit Dashboard

- Stage 1 — generate_data.py: Produces raw CSVs for customers, products, orders, and order items with intentional anomalies.
- Stage 2 — clean_data.py: Validates and cleans each dataset, cascades referential-integrity fixes across tables, and loads the warehouse.
- Stage 3 — SQL Warehouse: A normalized schema (customers, products, orders, order_items) with foreign-key constraints enforced.
- Stage 4a — report_cli.py: Runs data-integrity audits and prints/saves an executive KPI summary for a given date range.
- Stage 4b — Streamlit Dashboard: Five interactive pages — Executive Snapshot, Sales Overview, Customer Insights, Product Analytics, and Cohort Retention.

5. Data Generation with Intentional Inconsistencies

The generator produces four related datasets and deliberately injects realistic data-quality problems so the downstream cleaning pipeline has genuine issues to detect and resolve:

- Customers: ~2% of records contain malformed email addresses (missing '@' or invalid domain).
- Products: ~10% of names contain inconsistent casing and stray whitespace.
- Orders: ~5% of records have a missing customer_id, and ~5% use an alternate date format to exercise the date parser.
- Order Items: ~2% reference a non-existent order_id (orphaned rows), and ~3% contain negative quantities simulating returns or data-entry errors.

6. Data Cleaning & Validation (Pandas)

The cleaning pipeline (data_cleaner.py) applies a series of validation rules using Pandas and produces an auditable report of every row removed and why:

- Regex-based email validation to drop invalid customer records.
- String normalization (trim + title-case) on product names.
- Cascading referential-integrity checks: orders referencing a customer removed during cleaning are dropped; order_items referencing a dropped or missing order are removed.
- Multi-format date parsing with graceful fallback, dropping rows that remain unparseable.
- Zero-quantity order-item rows are removed while negative quantities (returns) are preserved for return-rate analysis.

7. Complex SQL Analytics

All analytical logic is centralized in a shared query module so the CLI and dashboard never diverge. The following SQL techniques were implemented:

- Joins — multi-table INNER JOINs across customers, orders, order_items, and products to compute revenue.
- Aggregations — SUM, COUNT, AVG for KPIs such as net revenue, order counts, and average line value.
- Window Functions — NTILE(4) for quartile-based customer lifetime-value segmentation (Platinum/Gold/Silver/Bronze) and ROW_NUMBER() for order-sequence analysis.
- CTEs — multi-step Common Table Expressions used to build cohort signup groups, monthly activity, and interval calculations before final aggregation.
- Cohort Analysis — a full monthly retention model tracking what percentage of each signup cohort continues to place orders N months after registration.

Revenue is computed consistently everywhere using the formula:

```
revenue = quantity × unit_price × (1 - discount_percent / 100)
```

8. Edge Case Handling & Robustness

The system includes automated data-integrity audits, run before every executive report is generated, to guarantee reliability:

- Discount percentage exceeding 100% — flagged as a data-quality violation.
- Zero-quantity order items — flagged and excluded from revenue calculations.
- Orphaned order_items referencing a non-existent order — flagged and excluded.
- Future-dated report ranges — detected and surfaced as a warning rather than silently processed.
- Empty result sets for a selected filter range — handled gracefully with an explicit 'no records found' message instead of an error.

9. Command-Line Reporting Tool

report_cli.py accepts a report type (daily/weekly/monthly) and a date range, runs the integrity audits, and prints a formatted executive summary including net revenue, order and customer counts, period-over-period trend comparison, and a top-3 products table — with an option to save the report to disk.

10. Interactive Dashboard — Results

The Streamlit dashboard provides five pages for interactive exploration of the warehouse data, all sharing a common dark enterprise theme, live date-range filtering, and Plotly-based visualizations. Screenshots of each page are presented below.

Figure 1 — Executive Snapshot (Home)

Landing page showing top-level KPIs (net revenue, total orders, active customers, average line value), the daily revenue trend, and the order status mix.

Figure 2 — Sales Overview

Revenue broken down by product category alongside a full order-status breakdown, with the same KPI cards for context.

Figure 3 — Customer Insights

Lifetime-value quartile segmentation (Platinum/Gold/Silver/Bronze) computed via the NTILE() window function, alongside customer-type distribution.

Figure 4 — Product Analytics

Top-N product leaderboard by revenue with an adjustable slider filter and category color-coding.

Figure 5 — Cohort Retention Heatmap

Monthly cohort retention heatmap showing the percentage of each signup cohort still placing orders N months after registration, built using CTE-based cohort analysis SQL.

11. Conclusion

This assignment successfully implements a complete, production-style e-commerce analytics pipeline: synthetic data generation with realistic inconsistencies, a validated ETL cleaning process, a normalized SQL warehouse with enforced referential integrity, advanced SQL analytics (joins, aggregations, window functions, CTEs, and cohort analysis), a robust CLI reporting tool with built-in data-integrity audits, and a polished interactive Streamlit dashboard. Automated tests (Pytest) verify correct row counts, anomaly removal, and query correctness end-to-end, confirming the system is reliable and ready for further extension.

12. Repository

The complete source code, documentation, and setup instructions are available at:

https://github.com/Raj-Kumar-Sharma/CEI-Assignment-8